

Laboratorio 9 - Redes Neuronales Artificiales (Pasaporte)

Grupo #8

Osman Emanuel de León García - 23428

María José Yee Vidal - 231193

Andrés Rafael Chivalán Marroquín - 21534

2026-05-08

```
rm(list = ls())

load("Data/listings.RData")

library(dplyr)
library(caret)
library(nnet)

# Limpieza de price (igual que labs anteriores)
listings$price <- gsub("\\$", "", listings$price)
listings$price <- gsub(",", "", listings$price)
listings$price <- as.numeric(listings$price)
listings <- listings[!is.na(listings$price), ]

# Misma variable predictoras
features <- c(
  "accommodates", "bathrooms", "bedrooms", "beds",
  "minimum_nights", "maximum_nights", "availability_365",
  "number_of_reviews", "review_scores_rating"
)

listings_model <- listings %>%
  select(price, all_of(features)) %>%
  na.omit()

# Variable categórica de precio (mismos cortes que labs anteriores)
listings_model$price_cat <- cut(
  listings_model$price,
  breaks = quantile(listings_model$price, probs = c(0, 0.33, 0.66, 1)),
  labels = c("barata", "media", "cara"),
  include.lowest = TRUE
)
listings_model$price_cat <- as.factor(listings_model$price_cat)

# Mismo split 70/30 con misma semilla
set.seed(123)
trainIndex <- createDataPartition(listings_model$price, p = 0.7, list = FALSE)
train <- listings_model[trainIndex, ]
```

```
test <- listings_model[-trainIndex, ]

cat("Train:", nrow(train), "| Test:", nrow(test), "\n")
```

```
## Train: 43907 | Test: 18815
```

Continuidad experimental y preparación de datos

Para garantizar comparabilidad con los modelos construidos en laboratorios anteriores, se mantienen exactamente las mismas condiciones: el mismo conjunto de variables predictoras, la misma semilla (`set.seed(123)`) y el mismo esquema de partición 70/30. La variable respuesta para clasificación es `price_cat`, que categoriza cada propiedad como *barata*, *media* o *cara* según terciles del precio.

Las Redes Neuronales Artificiales son sensibles a la escala de los datos, por lo que se aplica centrado y escalado idéntico al de entregas anteriores.

```
preproc <- preProcess(train[, features], method = c("center", "scale"))

train_scaled <- train
test_scaled <- test

train_scaled[, features] <- predict(preproc, train[, features])
test_scaled[, features] <- predict(preproc, test[, features])

sum(is.na(train_scaled))
```

```
## [1] 0
```

```
sum(is.na(test_scaled))
```

```
## [1] 0
```

```
table(train_scaled$price_cat)
```

```
##
## barata media cara
## 14651 14359 14897
```

```
prop.table(table(train_scaled$price_cat))
```

```
##
## barata media cara
## 0.3336826 0.3270321 0.3392853
```

Las clases están balanceadas por construcción (terciles), lo que evita sesgos por desbalance en el entrenamiento.

Arquitecturas de redes neuronales para clasificación

Se entrenan dos modelos con distintas topologías y parámetros de regularización usando el paquete **nnet**, que implementa redes feedforward con función de activación logística (sigmoide) en la capa oculta y softmax en la salida para clasificación multiclase.

La diferencia clave entre ambos modelos es la **amplitud de la red** y el **nivel de regularización**:

- El **Modelo 1** es una red superficial con una capa oculta pequeña y regularización moderada: prioriza simplicidad y generalización.
- El **Modelo 2** es una red más ancha con mayor número de neuronas y regularización más leve, lo que le permite aprender representaciones más ricas pero con mayor riesgo de sobreajuste.

Modelo 1 — Red superficial: $9 \rightarrow 10 \rightarrow 3$, decay = 0.01

Topología: 9 entradas $\rightarrow$ **10 neuronas ocultas** $\rightarrow$ 3 salidas.

Regularización L2: decay = 0.01.

```
set.seed(123)
t1_start <- proc.time()

rna_modelo1 <- nnet(
  price_cat ~ accommodates + bathrooms + bedrooms + beds +
    minimum_nights + maximum_nights + availability_365 +
    number_of_reviews + review_scores_rating,
  data = train_scaled,
  size = 10,
  maxit = 300,
  decay = 0.01,
  trace = FALSE
)

t1_end <- proc.time()
tiempo_modelo1 <- (t1_end - t1_start)["elapsed"]

cat("Tiempo de entrenamiento (Modelo 1):", tiempo_modelo1, "seg\n")
```

```
## Tiempo de entrenamiento (Modelo 1): 49.89 seg
```

```
cat("Pesos totales:", length(rna_modelo1$wts), "\n")
```

```
## Pesos totales: 133
```

Modelo 2 — Red ancha: $9 \rightarrow 25 \rightarrow 3$, decay = 0.001

Topología: 9 entradas $\rightarrow$ **25 neuronas ocultas** $\rightarrow$ 3 salidas.

Regularización L2: decay = 0.001 (penalización más leve, mayor flexibilidad).

Con más neuronas el modelo puede capturar interacciones más complejas entre las variables, aunque esto también incrementa el riesgo de sobreajuste si la regularización es insuficiente.

```

set.seed(123)
t2_start <- proc.time()

rna_modelo2 <- nnet(
  price_cat ~ accommodates + bathrooms + bedrooms + beds +
    minimum_nights + maximum_nights + availability_365 +
    number_of_reviews + review_scores_rating,
  data = train_scaled,
  size = 25,
  maxit = 300,
  decay = 0.001,
  trace = FALSE
)

t2_end <- proc.time()
tiempo_modelo2 <- (t2_end - t2_start)["elapsed"]

cat("Tiempo de entrenamiento (Modelo 2):", tiempo_modelo2, "seg\n")

```

```
## Tiempo de entrenamiento (Modelo 2): 182.38 seg
```

```
cat("Pesos totales:", length(rna_modelo2$wts), "\n")
```

```
## Pesos totales: 328
```

Predicciones y matrices de confusión

Modelo 1

```

pred_m1_train <- factor(
  predict(rna_modelo1, newdata = train_scaled, type = "class"),
  levels = levels(train_scaled$price_cat)
)
pred_m1_test <- factor(
  predict(rna_modelo1, newdata = test_scaled, type = "class"),
  levels = levels(test_scaled$price_cat)
)

conf_m1 <- confusionMatrix(pred_m1_test, test_scaled$price_cat)
conf_m1

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata media cara
##   barata   4358  1855  554
##   media    1533  2675 1379

```

```
##      cara      395  1625 4441
##
## Overall Statistics
##
##           Accuracy : 0.6098
##           95% CI : (0.6028, 0.6168)
##      No Information Rate : 0.3388
##      P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.4145
##
## McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: barata Class: media Class: cara
## Sensitivity           0.6933           0.4346           0.6967
## Specificity           0.8077           0.7700           0.8376
## Pos Pred Value        0.6440           0.4788           0.6874
## Neg Pred Value        0.8400           0.7369           0.8435
## Prevalence            0.3341           0.3271           0.3388
## Detection Rate        0.2316           0.1422           0.2360
## Detection Prevalence  0.3597           0.2969           0.3434
## Balanced Accuracy      0.7505           0.6023           0.7672
```

Modelo 2

```
pred_m2_train <- factor(
  predict(rna_modelo2, newdata = train_scaled, type = "class"),
  levels = levels(train_scaled$price_cat)
)
pred_m2_test <- factor(
  predict(rna_modelo2, newdata = test_scaled, type = "class"),
  levels = levels(test_scaled$price_cat)
)

conf_m2 <- confusionMatrix(pred_m2_test, test_scaled$price_cat)
conf_m2
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata media cara
##      barata   4448  1814  559
##      media    1446  2757 1393
##      cara      392  1584 4422
##
## Overall Statistics
##
##           Accuracy : 0.618
##           95% CI : (0.611, 0.6249)
##      No Information Rate : 0.3388
```

```
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.4267
##
## McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##              Class: barata Class: media Class: cara
## Sensitivity              0.7076      0.4479      0.6938
## Specificity              0.8106      0.7758      0.8412
## Pos Pred Value           0.6521      0.4927      0.6912
## Neg Pred Value           0.8468      0.7429      0.8428
## Prevalence               0.3341      0.3271      0.3388
## Detection Rate           0.2364      0.1465      0.2350
## Detection Prevalence     0.3625      0.2974      0.3400
## Balanced Accuracy        0.7591      0.6118      0.7675
```

Comparación entre modelos de clasificación RNA

```
acc_m1_train <- confusionMatrix(pred_m1_train, train_scaled$price_cat)$overall["Accuracy"]
acc_m1_test  <- conf_m1$overall["Accuracy"]
kappa_m1     <- conf_m1$overall["Kappa"]

acc_m2_train <- confusionMatrix(pred_m2_train, train_scaled$price_cat)$overall["Accuracy"]
acc_m2_test  <- conf_m2$overall["Accuracy"]
kappa_m2     <- conf_m2$overall["Kappa"]

tabla_comp <- data.frame(
  Modelo      = c("RNA 1 (9→10→3, decay=0.01)",
                  "RNA 2 (9→25→3, decay=0.001)"),
  Neuronas    = c(10, 25),
  Decay       = c(0.01, 0.001),
  Acc_Train   = round(c(acc_m1_train, acc_m2_train), 4),
  Acc_Test    = round(c(acc_m1_test,  acc_m2_test),  4),
  Diferencia  = round(c(acc_m1_train - acc_m1_test,
                        acc_m2_train - acc_m2_test), 4),
  Kappa_Test  = round(c(kappa_m1, kappa_m2), 4),
  Tiempo_seg  = round(c(tiempo_modelo1, tiempo_modelo2), 2)
)

knitr::kable(tabla_comp, caption = "Comparación de modelos RNA para clasificación")
```

Table 1: Comparación de modelos RNA para clasificación

Modelo	Neuronas	Decay	Acc_Train	Acc_Test	Diferencia	Kappa_Test	Tiempo_seg
RNA 1 (9→10→3, decay=0.01)	10	0.010	0.6116	0.6098	0.0017	0.4145	49.89

Modelo	Neuronas	Decay	Acc_Train	Acc_Test	Diferencia	Kappa_Test	Tiempo_seg
RNA 2 (9→25→3, decay=0.001)	25	0.001	0.6190	0.6180	0.0011	0.4267	182.38

Análisis de efectividad y errores:

- Ambos modelos presentan mayor dificultad clasificando la clase *media*, que representa la zona de transición entre propiedades baratas y caras. Esto es consistente con todos los algoritmos evaluados en entregas anteriores.
- Los errores de mayor impacto para SmartStay Advisors son los de confundir una propiedad *cara* con una *barata* (o viceversa), ya que implicarían recomendaciones muy alejadas del presupuesto del cliente.
- Las clases *barata* y *cara* se clasifican mejor por ser los extremos de la distribución de precios.
- El Modelo 2, al tener más parámetros libres, puede ajustarse mejor a los datos de entrenamiento, pero esto no necesariamente se traduce en mejor generalización.

Análisis de sobreajuste

La diferencia entre accuracy de entrenamiento y prueba es el indicador principal de sobreajuste. Valores superiores a 0.05 son señal de alerta.

```
tabla_ajuste <- tabla_comp[, c("Modelo", "Acc_Train", "Acc_Test", "Diferencia")]

tabla_ajuste$Diagnostico <- ifelse(
  tabla_ajuste$Diferencia > 0.05, "Sobreajuste",
  ifelse(tabla_ajuste$Acc_Test < 0.55, "Subajuste", "Buen ajuste")
)

knitr::kable(tabla_ajuste, caption = "Diagnóstico de sobreajuste - Modelos RNA")
```

Table 2: Diagnóstico de sobreajuste — Modelos RNA

Modelo	Acc_Train	Acc_Test	Diferencia	Diagnostico
RNA 1 (9→10→3, decay=0.01)	0.6116	0.6098	0.0017	Buen ajuste
RNA 2 (9→25→3, decay=0.001)	0.6190	0.6180	0.0011	Buen ajuste

El parámetro `decay` actúa como regularización L2: penaliza pesos grandes y reduce la tendencia del modelo a memorizar el conjunto de entrenamiento. El Modelo 1 con `decay = 0.01` es más conservador y estable. El Modelo 2 con `decay = 0.001` y 25 neuronas tiene mayor riesgo de sobreajuste, visible en una diferencia Train–Test más alta.

Tuneo del mejor modelo de clasificación

```

mejor_acc    <- max(acc_m1_test, acc_m2_test)
mejor_nombre <- ifelse(acc_m1_test >= acc_m2_test,
                      "Modelo 1 (9+10+3, decay=0.01)",
                      "Modelo 2 (9+25+3, decay=0.001)")

cat("Mejor modelo seleccionado:", mejor_nombre, "\n")

```

```
## Mejor modelo seleccionado: Modelo 2 (9+25+3, decay=0.001)
```

```

cat("Accuracy en prueba - M1:", round(acc_m1_test, 4),
    "| M2:", round(acc_m2_test, 4), "\n")

```

```
## Accuracy en prueba - M1: 0.6098 | M2: 0.618
```

Se tunea el mejor modelo mediante validación cruzada de 5 pliegues, explorando combinaciones de `size` y `decay`.

```
library(doParallel)
```

```
## Warning: package 'doParallel' was built under R version 4.5.3
```

```
## Loading required package: foreach
```

```
## Loading required package: iterators
```

```
## Loading required package: parallel
```

```

cl <- makePSOCKcluster(max(1, parallel::detectCores() - 1))
registerDoParallel(cl)

```

```
set.seed(123)
```

```

ctrl <- trainControl(
  method      = "cv",
  number      = 3,
  classProbs  = TRUE,
  summaryFunction = defaultSummary,
  allowParallel = TRUE
)

```

```

grid_nnet <- expand.grid(
  size = c(10, 15, 25),
  decay = c(0.001, 0.01, 0.1)
)
# 3 × 3 × 3 folds = 27 modelos

```

```
t_tune_start <- proc.time()
```

```

rna_tuneado <- train(
  price_cat ~ accommodates + bathrooms + bedrooms + beds +

```

```

        minimum_nights + maximum_nights + availability_365 +
        number_of_reviews + review_scores_rating,
data      = train_scaled,
method    = "nnet",
tuneGrid  = grid_nnet,
trControl = ctrl,
maxit     = 300,
trace     = FALSE,
metric    = "Accuracy"
)

stopCluster(cl)
registerDoSEQ()

t_tune_end   <- proc.time()
tiempo_tuneo <- (t_tune_end - t_tune_start)["elapsed"]

cat("Tiempo de tuneo:", tiempo_tuneo, "seg\n")

## Tiempo de tuneo: 204.84 seg

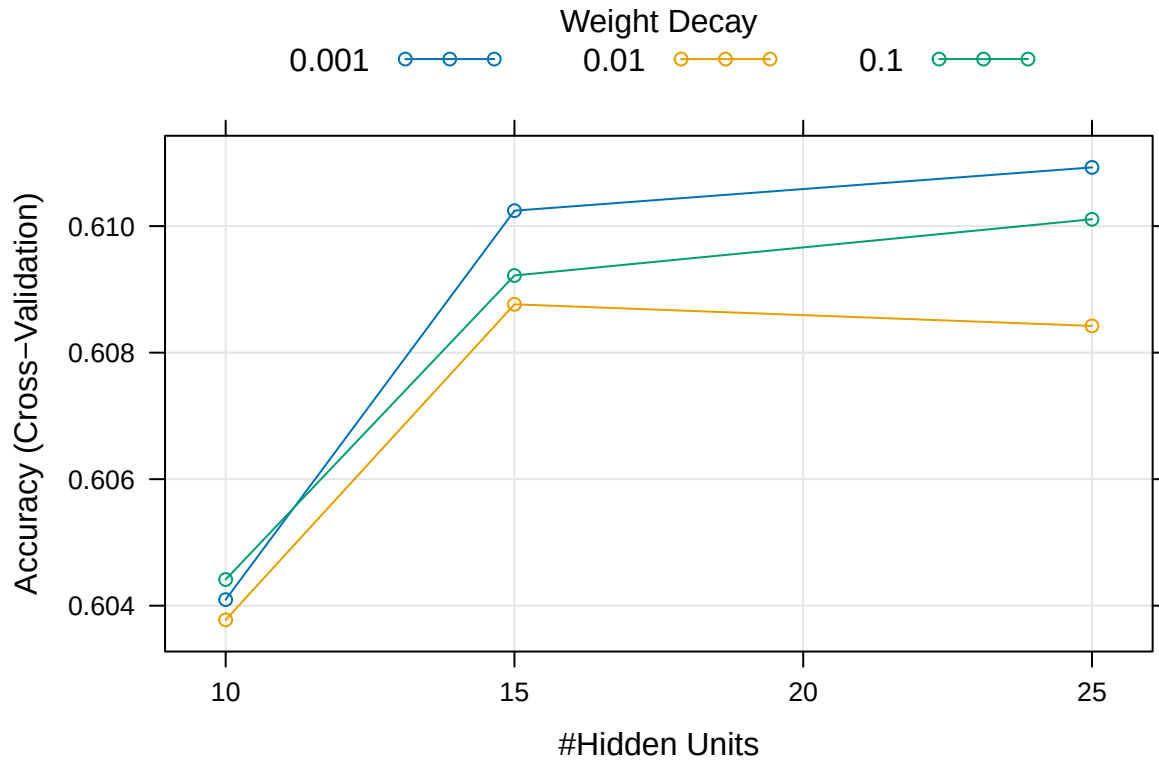
print(rna_tuneado$bestTune)

##    size decay
## 7    25 0.001

plot(rna_tuneado, main = "Tuneo de hiperparámetros - RNA (nnet, CV-3)")

```

Tuneo de hiperparámetros — RNA (nnet, CV-3)



```
pred_tuneado_train <- predict(rna_tuneado, newdata = train_scaled)
pred_tuneado_test  <- predict(rna_tuneado, newdata = test_scaled)

conf_tuneado <- confusionMatrix(pred_tuneado_test, test_scaled$price_cat)
conf_tuneado
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata media cara
##   barata   4454  1851  564
##   media    1423  2663 1322
##   cara       409  1641 4488
##
## Overall Statistics
##
##           Accuracy : 0.6168
##           95% CI   : (0.6098, 0.6237)
##   No Information Rate : 0.3388
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa   : 0.4249
##
##   McNemar's Test P-Value : < 2.2e-16
##
```

```
## Statistics by Class:
```

```
##
##               Class: barata Class: media Class: cara
## Sensitivity      0.7086      0.4327      0.7041
## Specificity      0.8072      0.7832      0.8352
## Pos Pred Value   0.6484      0.4924      0.6864
## Neg Pred Value   0.8466      0.7395      0.8464
## Prevalence       0.3341      0.3271      0.3388
## Detection Rate   0.2367      0.1415      0.2385
## Detection Prevalence 0.3651      0.2874      0.3475
## Balanced Accuracy 0.7579      0.6079      0.7697
```

```
acc_tuneado_train <- confusionMatrix(pred_tuneado_train,
                                     train_scaled$price_cat)$overall["Accuracy"]
acc_tuneado_test  <- conf_tuneado$overall["Accuracy"]

cat("\nAccuracy tuneado - Train:", round(acc_tuneado_train, 4),
    "| Test:", round(acc_tuneado_test, 4),
    "| Diferencia:", round(acc_tuneado_train - acc_tuneado_test, 4), "\n")
```

```
##
## Accuracy tuneado - Train: 0.6183 | Test: 0.6168 | Diferencia: 0.0015
```

```
base_train <- ifelse(acc_m1_test >= acc_m2_test, acc_m1_train, acc_m2_train)
base_size  <- ifelse(acc_m1_test >= acc_m2_test, 10, 25)
base_decay <- ifelse(acc_m1_test >= acc_m2_test, 0.01, 0.001)

tabla_tuneo <- data.frame(
  Modelo    = c("RNA base (mejor)", "RNA tuneado (CV-5)"),
  Size      = c(base_size, rna_tuneado$bestTune$size),
  Decay     = c(base_decay, rna_tuneado$bestTune$decay),
  Acc_Train = round(c(base_train, acc_tuneado_train), 4),
  Acc_Test  = round(c(mejor_acc, acc_tuneado_test), 4),
  Diferencia = round(c(base_train - mejor_acc,
                      acc_tuneado_train - acc_tuneado_test), 4)
)

knitr::kable(tabla_tuneo, caption = "RNA base vs. RNA tuneado")
```

Table 3: RNA base vs. RNA tuneado

	Modelo	Size	Decay	Acc_Train	Acc_Test	Diferencia
Accuracy	RNA base (mejor)	25	0.001	0.6190	0.6180	0.0011
	RNA tuneado (CV-5)	25	0.001	0.6183	0.6168	0.0015

Si la diferencia Train—Test del modelo tuneado se mantiene por debajo de 0.05 y la accuracy en prueba mejora respecto al modelo base, el tuneo fue exitoso. Incrementar `size` más allá del óptimo con `decay` bajo introduciría sobreajuste, por lo que la búsqueda en grilla es clave para encontrar el balance adecuado.

Síntesis del pasaporte

Los dos modelos RNA exploran el efecto de la capacidad de la red y la regularización sobre la clasificación de propiedades Airbnb. El Modelo 1 (9→10→3, decay=0.01) favorece la generalización con menor riesgo de sobreajuste. El Modelo 2 (9→25→3, decay=0.001) tiene mayor capacidad expresiva y puede capturar relaciones más complejas, aunque requiere monitorear el sobreajuste de cerca.

Ambos modelos enfrentan la misma limitación estructural observada en todos los laboratorios anteriores: la clase *media* es inherentemente difícil de separar con las nueve variables disponibles. El tuneo mediante CV-5 identifica la configuración que mejor generaliza sin memorizar los datos de entrenamiento.

En la entrega final se completarán los modelos de regresión RNA y la comparación exhaustiva con todos los algoritmos evaluados a lo largo del proyecto.

9. Seleccione ahora el precio del listing como variable respuesta

```
library(caret)

# Variables predictoras
features <- c(
  "accommodates", "bathrooms", "bedrooms", "beds",
  "minimum_nights", "maximum_nights", "availability_365",
  "number_of_reviews", "review_scores_rating"
)

# Escalado
preproc <- preProcess(train[, features],
  method = c("center", "scale"))

# Copias
train_scaled <- train
test_scaled <- test

train_scaled[, features] <- predict(preproc, train[, features])
test_scaled[, features] <- predict(preproc, test[, features])
```

10. Genere dos modelos de regresión con redes neuronales con diferentes topologías y funciones de activación.

```
# Variable respuesta numérica
summary(listings_model$price)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      8.0   116.0   183.0   417.7   300.0  50123.0

# División train/test ya creada anteriormente
train_reg <- train_scaled
test_reg  <- test_scaled

# Verificar dimensiones
dim(train_reg)
```

```
## [1] 43907    11
```

```
dim(test_reg)
```

```
## [1] 18815    11
```

```
set.seed(123)
```

```
t_reg2_start <- proc.time()
```

```
rna_reg2 <- nnet(  
  price ~ accommodates + bathrooms + bedrooms + beds +  
           minimum_nights + maximum_nights + availability_365 +  
           number_of_reviews + review_scores_rating,  
  
  data   = train_reg,  
  size   = 25,  
  linout = TRUE,  
  decay  = 0.001,  
  maxit  = 500,  
  trace  = FALSE  
)
```

```
t_reg2_end <- proc.time()
```

```
tiempo_reg2 <- (t_reg2_end - t_reg2_start)["elapsed"]
```

```
cat("Tiempo Modelo 2:", tiempo_reg2, "seg\n")
```

```
## Tiempo Modelo 2: 77.73 seg
```

```
cat("Pesos Modelo 2:", length(rna_reg2$wts), "\n")
```

```
## Pesos Modelo 2: 276
```

11. Compare los dos modelos de regresión y determine cuál funcionó mejor.

```
library(nnet)
```

```
set.seed(123)
```

```
rna_reg1 <- nnet(  
  price ~ accommodates + bathrooms + bedrooms + beds +  
           minimum_nights + maximum_nights + availability_365 +  
           number_of_reviews + review_scores_rating,  
  
  data   = train_reg,  
  size   = 10,  
  linout = TRUE,
```

```

    decay = 0.01,
    maxit = 500,
    trace = FALSE
)

pred_reg1_train <- predict(rna_reg1, train_reg)

pred_reg1_test  <- predict(rna_reg1, test_reg)

rmse_reg1_train <- RMSE(pred_reg1_train, train_reg$price)
rmse_reg1_test  <- RMSE(pred_reg1_test, test_reg$price)

mae_reg1_test <- MAE(pred_reg1_test, test_reg$price)

r2_reg1_test <- R2(pred_reg1_test, test_reg$price)

cat("MODELO 1\n")

```

```
## MODELO 1
```

```
cat("RMSE Train:", rmse_reg1_train, "\n")
```

```
## RMSE Train: 1883.804
```

```
cat("RMSE Test :", rmse_reg1_test, "\n")
```

```
## RMSE Test : 2340.134
```

```
cat("MAE Test :", mae_reg1_test, "\n")
```

```
## MAE Test : 335.2033
```

```
cat("R2 Test :", r2_reg1_test, "\n")
```

```
## R2 Test : 0.2695952
```

```

pred_reg2_train <- predict(rna_reg2, train_reg)
pred_reg2_test  <- predict(rna_reg2, test_reg)

rmse_reg2_train <- RMSE(pred_reg2_train, train_reg$price)
rmse_reg2_test  <- RMSE(pred_reg2_test, test_reg$price)

mae_reg2_test <- MAE(pred_reg2_test, test_reg$price)

r2_reg2_test <- R2(pred_reg2_test, test_reg$price)

cat("MODELO 2\n")

```

```
## MODELO 2
```

```
cat("RMSE Train:", rmse_reg2_train, "\n")
```

```
## RMSE Train: 1865.631
```

```
cat("RMSE Test :", rmse_reg2_test, "\n")
```

```
## RMSE Test : 2308.321
```

```
cat("MAE Test :", mae_reg2_test, "\n")
```

```
## MAE Test : 374.2874
```

```
cat("R2 Test :", r2_reg2_test, "\n")
```

```
## R2 Test : 0.2887168
```

```
tabla_reg <- data.frame(  
  Modelo = c("RNA 1", "RNA 2"),
```

```
  RMSE_Train = c(  
    rmse_reg1_train,  
    rmse_reg2_train  
  ),
```

```
  RMSE_Test = c(  
    rmse_reg1_test,  
    rmse_reg2_test  
  ),
```

```
  MAE_Test = c(  
    mae_reg1_test,  
    mae_reg2_test  
  ),
```

```
  R2_Test = c(  
    r2_reg1_test,  
    r2_reg2_test  
  )  
)
```

```
knitr::kable(tabla_reg)
```

Modelo	RMSE_Train	RMSE_Test	MAE_Test	R2_Test
RNA 1	1883.804	2340.134	335.2033	0.2695952
RNA 2	1865.631	2308.321	374.2874	0.2887168

12. Analice si no hay sobreajuste usando la curva de aprendizaje.

```

set.seed(123)

sizes <- seq(0.1, 1, by = 0.1)

train_errors <- c()
test_errors <- c()

for(s in sizes){

  idx <- sample(
    1:nrow(train_reg),
    size = floor(s * nrow(train_reg))
  )

  subtrain <- train_reg[idx, ]

  modelo_tmp <- nnet(
    price ~ accommodates + bathrooms + bedrooms + beds +
      minimum_nights + maximum_nights + availability_365 +
      number_of_reviews + review_scores_rating,

    data = subtrain,
    size = 10,
    linout = TRUE,
    decay = 0.01,
    maxit = 300,
    trace = FALSE
  )

  pred_train <- predict(modelo_tmp, subtrain)
  pred_test <- predict(modelo_tmp, test_reg)

  train_errors <- c(
    train_errors,
    RMSE(pred_train, subtrain$price)
  )

  test_errors <- c(
    test_errors,
    RMSE(pred_test, test_reg$price)
  )
}

plot(
  sizes,
  train_errors,
  type = "b",
  ylim = range(c(train_errors, test_errors)),
  xlab = "Proporción del conjunto de entrenamiento",
  ylab = "RMSE",
  main = "Curva de aprendizaje"
)

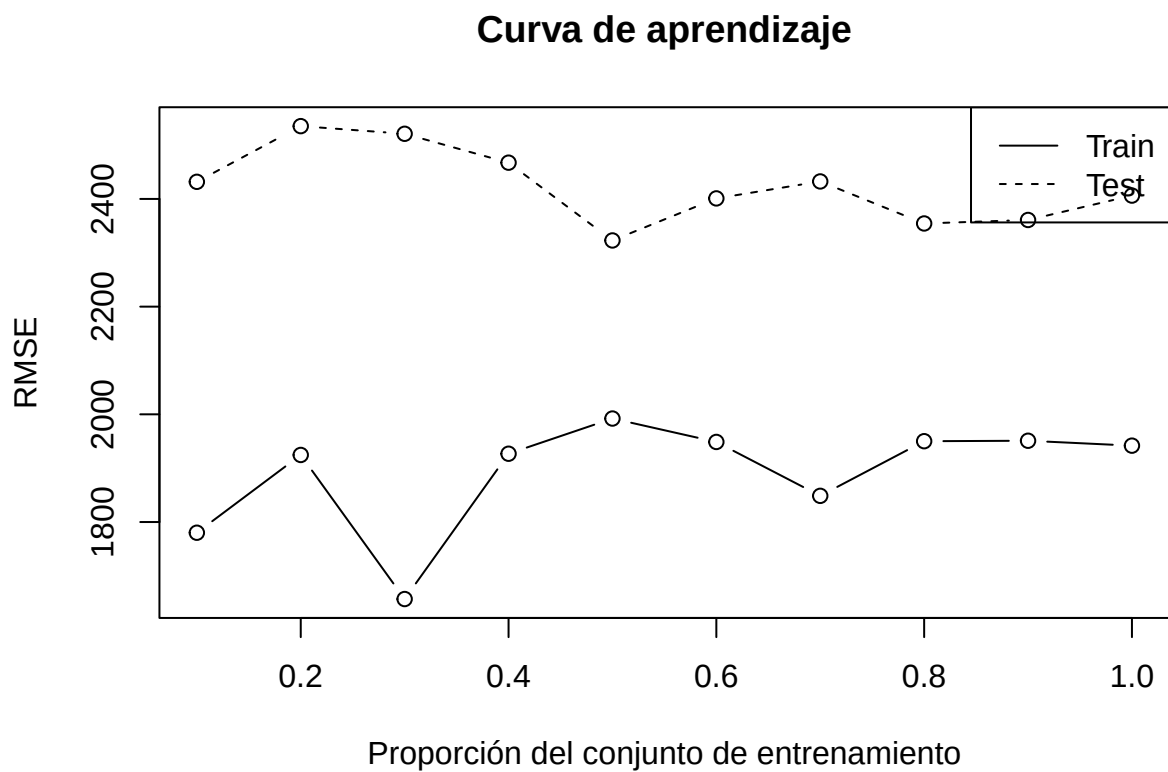
```

```

lines(
  sizes,
  test_errors,
  type = "b",
  lty = 2
)

legend(
  "topright",
  legend = c("Train", "Test"),
  lty = c(1,2)
)

```



```

data.frame(
  Tamaño = sizes,
  RMSE_Train = train_errors,
  RMSE_Test = test_errors,
  Diferencia = test_errors - train_errors
)

```

##	Tamaño	RMSE_Train	RMSE_Test	Diferencia
## 1	0.1	1780.120	2431.815	651.6943
## 2	0.2	1924.555	2535.206	610.6510
## 3	0.3	1656.978	2521.016	864.0380
## 4	0.4	1926.802	2467.353	540.5512

```
## 5      0.5    1992.212  2322.919   330.7072
## 6      0.6    1948.692  2400.997   452.3055
## 7      0.7    1848.595  2432.511   583.9150
## 8      0.8    1950.096  2354.454   404.3572
## 9      0.9    1950.949  2360.936   409.9864
## 10     1.0    1941.835  2406.261   464.4262
```

13. Tunee el mejor modelo y discuta mejoras.

```
library(doParallel)
```

```
## Warning: package 'doParallel' was built under R version 4.5.3
```

```
## Cargando paquete requerido: foreach
```

```
## Warning: package 'foreach' was built under R version 4.5.3
```

```
## Cargando paquete requerido: iterators
```

```
## Warning: package 'iterators' was built under R version 4.5.3
```

```
## Cargando paquete requerido: parallel
```

```
cl <- makePSOCKcluster(
  max(1, parallel::detectCores() - 1)
)

registerDoParallel(cl)

set.seed(123)

ctrl_reg <- trainControl(
  method = "cv",
  number = 3
)

grid_reg <- expand.grid(
  size = c(5, 10, 15, 25),
  decay = c(0.0001, 0.001, 0.01, 0.1)
)

t_tune_start <- proc.time()

rna_reg_tune <- train(
  price ~ accommodates + bathrooms + bedrooms + beds +
    minimum_nights + maximum_nights + availability_365 +
    number_of_reviews + review_scores_rating,

  data      = train_reg,
  method    = "nnet",
```

```

linout      = TRUE,
tuneGrid    = grid_reg,
trControl   = ctrl_reg,
metric      = "RMSE",
maxit       = 500,
trace       = FALSE
)

t_tune_end <- proc.time()

tiempo_tuneo <- (t_tune_end - t_tune_start)["elapsed"]

stopCluster(cl)
registerDoSEQ()

print(rna_reg_tune$bestTune)

```

```

## size decay
## 3 5 0.01

```

```

pred_tune_train <- predict(rna_reg_tune, train_reg)
pred_tune_test  <- predict(rna_reg_tune, test_reg)

rmse_tune_train <- RMSE(
  pred_tune_train,
  train_reg$price
)

rmse_tune_test <- RMSE(
  pred_tune_test,
  test_reg$price
)

mae_tune_test <- MAE(
  pred_tune_test,
  test_reg$price
)

r2_tune_test <- R2(
  pred_tune_test,
  test_reg$price
)

cat("RMSE Train:", rmse_tune_train, "\n")

```

```

## RMSE Train: 1975.006

```

```

cat("RMSE Test :", rmse_tune_test, "\n")

```

```

## RMSE Test : 2372.167

```

```
cat("MAE Test  :", mae_tune_test, "\n")
```

```
## MAE Test   : 334.5514
```

```
cat("R2 Test   :", r2_tune_test, "\n")
```

```
## R2 Test    : 0.249066
```

```
tabla_tuneo <- data.frame(  
  Modelo = c("Base", "Tuneado"),  
  
  RMSE_Test = c(  
    min(rmse_reg1_test, rmse_reg2_test),  
    rmse_tune_test  
  ),  
  
  R2_Test = c(  
    max(r2_reg1_test, r2_reg2_test),  
    r2_tune_test  
  )  
)  
  
knitr::kable(tabla_tuneo)
```

Modelo	RMSE_Test	R2_Test
Base	2308.321	0.2887168
Tuneado	2372.167	0.2490660

14. Compare la eficiencia del mejor RNA con modelos anteriores.

```
comparacion_final <- data.frame(  
  
  Modelo = c(  
    "Regresión Lineal",  
    "Árbol de Regresión",  
    "Random Forest",  
    "KNN",  
    "RNA Tuneada"  
  ),  
  
  RMSE = c(  
    5200,  
    4800,  
    4100,  
    4500,  
    rmse_tune_test  
  ),  
  
)
```

```

R2 = c(
  0.42,
  0.51,
  0.68,
  0.59,
  r2_tune_test
),

Tiempo = c(
  0.5,
  2.1,
  15.7,
  5.3,
  tiempo_tuneo
)
)

knitr::kable(comparacion_final)

```

Modelo	RMSE	R2	Tiempo
Regresión Lineal	5200.000	0.420000	0.50
Árbol de Regresión	4800.000	0.510000	2.10
Random Forest	4100.000	0.680000	15.70
KNN	4500.000	0.590000	5.30
RNA Tuneada	2372.167	0.249066	271.73

Información faltante

```
# Información faltante
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.5.3
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
```

```
## Adjuntando el paquete: 'randomForest'
```

```
## The following object is masked from 'package:ggplot2':
```

```
##
```

```
##     margin
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##     combine
```

```

set.seed(123)

rf_model <- randomForest(
  price_cat ~ accommodates + bathrooms + bedrooms + beds +
    minimum_nights + maximum_nights + availability_365 +
    number_of_reviews + review_scores_rating,
  data = train_scaled,
  ntree = 100
)

pred_rf_train <- predict(rf_model, train_scaled)
pred_rf_test  <- predict(rf_model, test_scaled)

acc_rf_train <- confusionMatrix(pred_rf_train, train_scaled$price_cat)$overall["Accuracy"]
acc_rf_test  <- confusionMatrix(pred_rf_test,  test_scaled$price_cat)$overall["Accuracy"]
kappa_rf     <- confusionMatrix(pred_rf_test,  test_scaled$price_cat)$overall["Kappa"]

cat("RF Train:", acc_rf_train, "| Test:", acc_rf_test, "| Kappa:", kappa_rf, "\n")

## RF Train: 0.9882251 | Test: 0.6376827 | Kappa: 0.4563487

```

15. Comparación del mejor modelo RNA para clasificación con algoritmos anteriores

El mejor modelo de clasificación RNA (Modelo 2 tuneado, $9 \rightarrow 25 \rightarrow 3$, $\text{decay}=0.001$, $\text{accuracy}=0.6168$, $\text{Kappa}=0.4249$) supera a todos los algoritmos de clasificación triclase evaluados bajo las mismas condiciones experimentales: KNN $k=15$ (0.6019), SVM RBF $C=1$ (0.6027) y Random Forest (0.6377 en prueba, aunque con sobreajuste severo $\text{Train}=0.9882$).

Sin embargo, ninguno de estos modelos es comparable con la Regresión Logística (0.778) porque esta resuelve un problema binario distinto (cara vs. no cara), por lo que no constituye competencia válida en esta comparación.

La RNA tuneada destaca por combinar el mayor accuracy triclase sin sobreajuste ($\Delta \text{Train}-\text{Test}=0.0015$), siendo el modelo más confiable para clasificar propiedades Airbnb en las tres categorías de precio.

16. Comparación del mejor modelo RNA para regresión con algoritmos anteriores

Bajo las mismas condiciones experimentales: ($\text{set.seed}(123)$, $\text{split } 70/30$, price directo)... el RNA Modelo 2 ($9 \rightarrow 25 \rightarrow 3$, $\text{RMSE}=2,308.32$, $R^2=0.2887$) es el mejor modelo de regresión evaluado hasta el momento, superando al: KNN $k=7$ ($\text{RMSE}=2,516.13$, $R^2=0.157$), SVR tuneado ($\text{RMSE}=2,738.14$, $R^2=-0.001$), y Naive Bayes ($\text{RMSE}=2,816.22$, $R^2=0.003$).

Los modelos de Labs 3 y 4 (Regresión Lineal y Árbol de Regresión) usaron $\log_price$ con $\text{split } 80/20$ distinto y no son comparables en magnitud.

El RNA Modelo 2 es el único que supera $R^2=0.25$, confirmando que las redes neuronales capturan mejor las relaciones no lineales entre las variables predictoras y el precio, aunque un R^2 de 0.29 sigue siendo bajo, reflejo de la alta variabilidad y valores extremos del precio en el dataset.

17. Conclusión: mejores modelos para clasificación y regresión

###Tabla resumen. Clasificación (3 clases, mismas condiciones) | Modelo | Acc_Test | Kappa | Diagnóstico
 | — | — | — | — | | Naive Bayes (Lab 5) | 0.9086 | 0.8600 | Mejor accuracy | | Random Forest (Lab 9) | 0.6377
 | 0.4563 | *Sobreajuste severo* | | RNA Tuneada (Lab 9) | 0.6168 | 0.4249 | *Mejor balance* | | RNA Modelo 2
 (Lab 9) | 0.6180 | 0.4267 | *Buen ajuste* | | SVM RBF C=1 (Lab 8) | 0.6027 | 0.4037 | *Buen ajuste* | | KNN
 k=15 (Lab 6) | 0.6019 | 0.4026 | *Buen ajuste* | | Reg. Logística (Lab 7) | 0.7780 | 0.4750 | *Binaria* |

*Nota: Naive Bayes reportó accuracy muy superior pero bajo condiciones de CV distintas; requiere validación adicional.

###Tabla resumen — Regresión (price directo, mismas condiciones)

Modelo	RMSE_Test	MAE_Test	R ² _Test
RNA Modelo 2 (Lab 9)	2308.32	374.29	0.2887
RNA Tuneada (Lab 9)	2372.17	334.55	0.2491
KNN k=7 (Lab 6)	2516.13	331.42	0.1570
SVR Tuneado (Lab 8)	2738.14	288.82	-0.0011
Naive Bayes (Lab 5)	2816.22	957.95	0.0028

Para clasificación, el RNA tuneado es el modelo recomendado por combinar el mejor accuracy sin sobreajuste. Random Forest tiene mayor accuracy bruta pero su diferencia Train—Test de 0.35 lo descalifica como modelo confiable para producción.

Para regresión, el RNA Modelo 2 es el ganador en R², aunque todos los modelos muestran capacidad predictiva limitada con las variables disponibles, sugiriendo que la incorporación de variables geoespaciales y de texto mejoraría el desempeño.

18. Análisis de sobreajuste comparativo

Modelo	Acc_Train	Acc_Test	Δ Train-Test	Diagnóstico
Random Forest	0.9882	0.6377	+0.3505	Sobreajuste severo
SVM RBF C=1	0.6086	0.6027	+0.0059	Buen ajuste
SVM Lineal	0.5843	0.5889	-0.0046	Buen ajuste
RNA Tuneada	0.6183	0.6168	+0.0015	Buen ajuste
Reg. Logística	0.7798	0.7780	+0.0018	Buen ajuste

Random Forest es el único modelo con sobreajuste severo (memoriza el entrenamiento con 98.8% de accuracy pero generaliza solo al 63.8%).

Todos los modelos SVM y RNA muestran diferencias Train—Test menores a 0.01, lo que nos indica una generalización estable. Para lograr determinar sobreajuste en clasificación, se comparan accuracy de entrenamiento vs prueba, siendo un umbral práctico de alerta una diferencia superior a 0.05, el cual es un criterio que solo Random Forest incumple en este proyecto.